

VLSI 网表二分划分实验报告

组号：6 陈奕航 23336047 陈泽义 23336050 陈俊鑫 23336040

1. 算法逻辑与实现思路

1.1 问题定义

给定超图 $G = (V, E)$ ，将节点集 V 划分为 X 和 Y 两个子集，在满足 2% 面积平衡约束的前提下最小化被切割的超边数。该问题属 NP-Hard，本实验采用多层次超图划分 + 改进 FM 细化的启发式框架求解。

1.2 数据结构：双向索引超图

算法内部使用紧凑的 Hypergraph 结构替代框架的指针式图表示：

```
nets[i]          // 线网 i 上的所有节点（内部索引）
node_nets[u]     // 节点 u 关联的所有线网
node_weight      // 节点权重（粗化时累加，原始为 1）
```

设计要点：同时维护"线网 -> 节点"和"节点 -> 线网"两组邻接关系。FM 增量更新时，仅需通过 $node_nets[u] \rightarrow nets[net_idx]$ 两跳定位受影响邻居，避免全图扫描；使用时间戳 $dedup_stamp$ 对邻居去重，保证每条边只处理一次。

1.3 总体流程（多次重启 + V-cycle）

```
重复 8 次（随机种子不同）：
├─ 粗化循环（最多 5 层）
│   └─ 重复调用 coarsen_once，直到节点 ≤ 320 或缩减率 > 0.92
├─ 最粗层初始划分（3 种策略竞争）
├─ FM 细化 + 打磨（粗层参数）
└─ 逐层反粗化
    └─ 每层投影划分后运行 FM（中层/细层参数 + 打磨）
保留割边数最小的解
```

外层设有 90 秒总时间预算，在每次循环、每层粗化、每轮 FM 中细粒度检查超时，超时即返回当前最优解。

1.4 粗化：带奖励与惩罚的匹配评分

每次粗化将节点两两配对成组。匹配评分机制：

$$score(u, v) = \sum_{\text{共享线网}} \frac{net_weight}{deg_penalty}$$

- 线网权重** = $1/(|net| - 1)$ ：线网越小权重越高，强化紧密耦合
- 小网奖励** ($|net| \leq 4$)：权重 $\times 1.7$ ，保护最常见的小线网不被割裂
- 度数惩罚** = $1 + 0.06 \times deg(v)$ ：抑制与"超级枢纽"节点匹配，避免形成权重过大的粗化节点

- **同分情况：** 优先选度数更小的节点，保持粗化图结构均衡

组内成员权重求和得到粗化节点权重，原线网投影去重后保留。

1.5 三种初始划分策略

在最粗层运行三种策略，FM 优化后择优：

策略	做法	侧重点
度数贪心	按度数降序，贪心放入较轻侧	高度数节点优先决策，初始解质量较好
随机贪心	随机打乱，贪心放入较轻侧	提供多样性，不同起点避免局部最优
小网权重	按小网亲和度降序，同时平衡两侧累计分数	与割边优化目标直接对齐

1.6 FM 细化：最大堆 + 延迟更新

采用改进的 FM 变体，核心机制：

增量增益更新： 节点移动后仅重新计算受影响邻居的增益（通过共享线网确定范围），时间戳去重避免重复计算。非邻居节点增益不变，无需全图扫描。

最大堆 + 延迟更新： 使用 `priority_queue<GainEntry>` 维护候选移动，`GainEntry` 含 `{gain, node, version}`。节点增益变化时递增 `version` 并推入新条目；堆顶弹出时若版本不匹配则丢弃。避免了每步 $O(n)$ 重建堆的开销。

平衡约束过滤： 弹出后检查移动是否满足 $[W_{\min}, W_{\max}]$ 约束，不满足则暂存至 `deferred` 数组稍后重新入堆。

使用全量扫描兜底： 若堆取样范围内无合法候选，退化执行 $O(n)$ 全扫描，确保不会遗漏可用移动。

最佳回滚： 每轮结束后回退到累计增益最大的移动前缀，舍弃后续无益移动。若全程无正增益则全部回退。

1.7 分层参数配置

参数	粗层	中层	细层（原始图）
FM 轮数	8	8	10
每轮最大步数	9000	10000	14000
堆检查上限	1800	2200	3000
打磨轮数	2	2	3
打磨最大步数	2500	3000	4500

细层（level 0）是原始图，配置最激进参数以充分挖掘优化潜力；粗层节点少参数保守，快速通过。每层主 FM 后追加**打磨轮**，试探被主轮遗漏的增益机会。

2. 实验结果

测试环境：C++17，Apple clang，90 秒总时间预算， $\epsilon = 2\%$ 。参考最优解取自 ISPD 基准数据集中 $\epsilon = 2\%$ 列。

Benchmark	节点数	线网数	本文 Cut	最优 ($\epsilon=2\%$)	绝对差	相对差
IBM01	12,752	14,111	261	200	61	30.50%
IBM02	19,601	19,584	355	307	48	15.64%
IBM03	23,136	27,401	1074	951	123	12.93%
IBM04	27,507	31,970	639	573	66	11.52%
IBM05	29,347	28,446	1848	1706	142	8.32%
IBM06	32,498	34,826	1074	962	112	11.64%
IBM07	45,926	48,117	1044	878	166	18.91%
IBM08	51,309	50,513	1219	1140	79	6.93%
IBM09	53,395	60,902	1035	620	415	66.94%
IBM10	69,429	75,196	1582	1253	329	26.26%
IBM11	70,558	81,454	1368	1051	317	30.16%
IBM12	71,076	77,240	2066	1919	147	7.66%
IBM13	84,199	99,666	967	831	136	16.37%
IBM14	147,605	152,772	3468	1842	1626	88.27%
IBM15	161,570	186,608	5595	2730	2865	104.95%
IBM16	183,484	190,048	5705	1827	3878	212.26%
IBM17	185,495	189,581	3321	2270	1051	46.30%
IBM18	210,613	201,920	5479	1521	3958	260.22%

整体平均相对差：54.21%

3. 实验总结

3.1 优点

- 1. **架构清晰**：多层次粗化-细化流程完整，粗化-初始化-细化 三模块解耦良好，便于替换和迭代改进。
- 2. **中小规模表现良好**：IBM02-IBM08、IBM12-IBM13 相对差在 7%-19%，IBM05（8.32%）和 IBM08（6.93%）尤为突出，验证了 多层次 + 增量 FM 路线的正确性。
- 3. **粗化质量**：小网加权 + 度数惩罚 + 同分度数优先的评分设计有效避免了超级节点过早形成，保护了图的结构信息。
- 4. **多样性搜索**：三种互补的初始划分策略以及 8 次随机重启，增大了对解空间的覆盖。

3.2 缺点

- 1. **大规模实例退化严重**：IBM14-IBM18（节点 > 14 万）相对差 88%-260%，主要因粗化质量不足、90 秒内细化不充分、重启多样性有限。
- 2. **堆结构短板**：最大堆 pop 为 $O(\log n)$ ，在大图上过期条目累积增加无效 pop 次数；不如 bucket-FM 的 $O(1)$ 查找高效。
- 3. **参数固定**：所有实例共用一套参数，未按图规模自适应调整粗化层数、FM 轮数等。

4. **粗化匹配局部性**：仅基于共享线网的局部评分，未考虑全局结构（如超边分布、社区结构），粗化图可能偏离最优搜索空间。

3.3 改进方向

1. **自适应参数**：按输入图规模（节点数、平均度数）动态调整重启数、粗化层数、FM 步数和候选范围。大图适当减少单次重启的 FM 开销，换取更多重启次数。
2. **更深层次粗化**：对大规模图增加到 7-8 层粗化，使最粗层充分缩小以降低细化难度。
3. **结构感知匹配**：在粗化评分中引入超边分布熵或社区检测信息，提升匹配的全局质量。
4. **并行加速**：FM 增量更新的邻居遍历天然适合多线程并行，可在细层上显著提速。